

Moje upoznavanje sa *praktičnom* primenom optimizacije u Pythonu

Poglavlje 1: Prvi susret

Kada sam počeo da istražujem o problemima optimizacije, prvo sam se susreo sa jednim od osnovnih optimizacionih problema, jednostavno raspoređivanje resursa (Production Mix), glavna ideja: kako uz minimalne troškove ili maksimalan učinak ispuniti pravila koja nam postavlja realan sistem, ta ideja se zadržala i u drugom problemu sa kojim sam se susreo, samo na složenijem sistemu dnevnog upravljanja energijom u modernom data centru (ideja kako bi se rešio ovaj problem, naravno na dosta jednostavnijem primeru i sa izmišljenim varijablama).

1. Šta sam naučio na osnovnom primeru: Logika solvera i "Infeasibility"?

([github](#): [test_opt.py](#))

Prvi izazov bio je klasični problem proizvodnje nameštaja koristeći **Google OR-Tools** (CP-SAT).

- **Kako funkcioniše:** Definišemo promenljive (`NewIntVar`, u određenom opsegu) kao odluke koje sistem donosi, postavimo stroga pravila, ograničenja (`model.Add`) i zadamo ciljnu funkciju (`Maximize` ili `Minimize`).
- **Glavna lekcija:** Kada sam eksperimentisao sa promenom znakova i tražio minimum uz nepromišljene uslove, solver je izbacio "Infeasible". Istražio sam šta to znači i naučio da su uslovi koje sam zadao međusobno kontradiktorni i da rešenje ne postoji.
- **CP-SAT:** Umesto da računar nasumično proba svaku kombinaciju, CP-SAT koristi fazu širenja uslova da brzo odbaci sve nemoguće opcije pre nego što krene u pretragu.

2. Prelazak na viši nivo: Matrično linearno programiranje i moj rad na sistemu za "Green Data Center"

([github](#): [green.py](#), [green_res.py](#))

Sledeći korak je bio prelazaki na realan industrijski problem: optimizacija potrošnje električne energije, rada servera i upravljanje baterijama u jednom velikom AI Data Centru kroz period od 24 sata. Ovde sam shvatio upotrebu matričnih sistema u softverskom inženjerstvu.

Prvo sam pokušao ručno da pišem ograničenja za više sati, loša odluka, zadržao sam se na tome nepotrebno dugo, amaterska greška. Kada imamo 24 sata i u svaki sat uvodimo više odluka (struja iz mreže za servere, pražnjenje baterije, punjenje baterije), broj promenljivih se znatno povećava. Ručno kucanje redova u matrici me dovodilo do dosta grešaka u indeksima koje je, bar meni, nemoguće ispratiti.

Rešenje: NumPy i SciPy

Rešenje je bilo u tome da iskoristim Python da sam generiše matricu koristeći funkcije biblioteke **NumPy**:

- **np.eye(hours)**: Shvatio sam kako se preko jedinične matrice uspostavlja "jedan na jedan" veza za svaki sat (gde sat **h** utiče samo na sopstveni energetski balans).
- **np.zeros**: Naučio sam da koristim nulte blokove da „popunim rupe“ tamo gde određena promenljiva (npr. punjenje baterije) ne učestvuje u nekom specifičnom ograničenju.
- **np.hstack** i **np.vstack**: Ovi alati su mi postali glavni oslonac za horizontalno spajanje promenljivih u širinu (svih kolona) i vertikalno slaganje različitih tipova ograničenja jedno ispod drugog (zahtevi servera, kapacitet baterija) u jedinstvenu formulaciju linearnog programiranja oblika $c^T x$ uz ograničenja $Ax \leq b$.

Izvršavanje i rezultat

Kada se ovako složena matrica prosledi **SciPy-jevom linprog solveru** (sa HiGHS algoritmom), sistem jako brzo izračunava globalni optimum. Analizirajući izlazne podatke (**res.x**), tačno vidim isplativost cele operacije: u kom satu se baterija puni po niskoj tarifi, kada se prazni da pokrije energetske skokove servera, i koliki je minimalni ukupni trošak električne energije.

Jedna od ideja ovog problema jeste bila procena isplativosti uvođenja “zelene” energije u data centre. S obzirom da data centri koriste mnogo električne energije, ideja je bila uvesti solarne panele koji bi bili bolji za životnu sredinu i “olakšavali” elektroenergetsku mrežu, i potencijalno smanjili troškove korišćenja električne energije. Što direktno vodi ka energetskej efikasnosti i održivosti.

Zaključak

Kroz ove zadatke shvatio sam da optimizacija od mene ne traži samo da pišem petlje, već da fizičke i ekonomske zakone iz stvarnog sveta prevedem na jezik matrica. Prelazak sa jednostavnih uslova na matrične sisteme dao mi je sigurnost da mogu programski da rešavam složene inženjerske probleme sa desetinama zavisnih promenljivih. Shvatio sam da je optimizacija veoma važna oblast, koja može da dovodi do napretka ne samo softvera već i čitave industrije i zaštite životne sredine.

Repozitorijum (primeri u kodu): <https://github.com/marovichn/python-optimization>

Literatura: https://developers.google.com/optimization/cp/cp_solver

<https://or.stackexchange.com/questions/tagged/optimization>

<https://github.com/google/or-tools/tree/stable/examples/python>

<https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.linprog.html>

<https://www.geeksforgeeks.org/python/maximize-optimization-using-scipy/>

<https://www.w3schools.com/python/numpy/default.asp>

Poglavlje 2: Plan Istraživanja

Nakon što sam se upoznao sa ova dva problema, prešao sam na potražnju najbolje literature za svoja interesovanja, iz ove literature ću istraživati teorijske osnove kao i praktične primere u python kodu po potrebi problema, kreirao sam i konkretan plan istraživanja po fazama (oblastima):

Literatura:

Osnove i modeliranje

- H. P. Williams, ***Model Building in Mathematical Programming*** (izbor modela na osnovu realnog problema)
- Hillier & Lieberman, ***Introduction to Operations Research*** (LP, mreže, transport, dodela)
- Dominik Krupke, ***CP-SAT Primer*** (Python primeri na github-u)
- Zvanična OR-Tools dokumentacija

Linearno programiranje (teorija)

- Bertsimas & Tsitsiklis, ***Introduction to Linear Optimization*** (Teorijska literatura)
- Robert Vanderbei, ***Linear Programming: Foundations and Extensions*** (Python code)

Celobrojno i kombinatorno programiranje

- Laurence Wolsey, ***Integer Programming***
- Conforti, Cornuéjols, Zambelli, ***Integer Programming*** (dublja teorija)
- Ahuja, Magnanti, Orlin, ***Network Flows*** (najkompletnija knjiga o tokovima, najkraćim putevima i dodeli)
- Korte & Vygen, ***Combinatorial Optimization*** (za dublje razumevanje algoritama)

Konveksna i nelinearna optimizacija

- Boyd & Vandenberghe, ***Convex Optimization*** (+ Stanford kurs **EE364a** na YouTube-u)
- Nocedal & Wright, ***Numerical Optimization*** (standard za nelinearnu optimizaciju)
- Kochenderfer & Wheeler, ***Algorithms for Optimization*** (Julia kod, može se lako implementirati u python)

Specijalizovane oblasti

- Pinedo, *Scheduling: Theory, Algorithms, and Systems* (raspoređivanje)
- Talbi, *Metaheuristics: From Design to Implementation* (genetski algoritmi, simulirano kaljenje, tabu pretraga)
- Birge & Louveaux, *Introduction to Stochastic Programming* (optimizacija pod neizvesnošću)

Plan:

Faza 1: Utvrđivanje LP

- Dualnost, komplementarna labavost, analiza osetljivosti (shadow prices, reduced costs).
- Čitaj: Bertsimas poglavlja 1–5, ili Vanderbei.
- Praksa: izvuci dualne promenljive iz **linprog** (HiGHS) i protumači ih ekonomski; problemi: ishrana, proizvodni miks, transport.

Faza 2: Modeliranje i MIP

- Binarne promenljive, logički uslovi, big-M, fiksni troškovi, "ili-ili" ograničenja.
- Čitaj: Williams (glavni resurs), Wolsey uvod.
- Praksa: **scipy.optimize.milp**, zatim OR-Tools/Pyomo. Probleme: ranac, facility location, lot-sizing, set cover.

Faza 3: CP-SAT i raspoređivanje

- Interval promenljive, **NoOverlap**, **AddCircuit**, optimizacija vs. izvodljivost.
- Čitaj: CP-SAT Primer, OR-Tools dokumentacija.
- Praksa: raspored smena (nurse rostering), job-shop, TSP/VRP sa vremenskim prozorima.

Faza 4: Mreže i grafovi

- Najkraći put, maksimalni tok, minimalni trošak toka, dodela.
- Čitaj: Ahuja izabrana poglavlja.
- Praksa: OR-Tools **SimpleMinCostFlow**, **networkx**.

Faza 5: Konveksna i nelinearna optimizacija

- Konveksnost, KKT uslovi, gradijentni spust, Newton, kvadratno programiranje.
- Gledaj: EE364a predavanja; čitaj Boyd poglavlja 1–5.
- Praksa: CVXPY (portfolio optimizacija, najmanji kvadrati sa ograničenjima), **`scipy.optimize.minimize`**.

Faza 6: Heuristike i napredne teme

- Metaheuristike za probleme kad egzaktno ne ide (simulirano kaljenje, genetski algoritmi), uvod u stohastičku optimizaciju.
- Zaključni projekat: izaberi realan problem (raspored, rutiranje, planiranje proizvodnje), modeliraj ga na dva načina (npr. MIP i CP-SAT) i uporedi vreme i kvalitet rešenja.

Poglavlje 3: Faza 1 - Utvrđivanje linearnog programiranja

1. Teorijski uvod

Standardni oblik

Primalni problem (P): $\min c^T x$, uz $Ax = b, x \geq 0$

Svaki LP ($\leq, \geq$, slobodne promenljive) može se prevesti u ovaj oblik pomoću **dopunskih promenljivih** i razdvajanja slobodne promenljive na $x = x^+ - x^-$.

Geometrija i simpleks

- Dopustiva oblast je **poliedar**. Ako optimum postoji, nalazi se u nekom **temenu** (ekstremnoj tački).
- Teme odgovara **baznom dopustivom rešenju (BDR)**: biramo m linearno nezavisnih kolona matrice A (baza B), ostale promenljive su 0, a $x_B = B^{-1}b \geq 0$.
- **Simpleks** se kreće sa temena na susedno teme koje poboljšava cilj. Zaustavlja se kad su svi **reducirani troškovi** $\bar{c}_j = c_j - c_B^T B^{-1} A_j \geq 0$ (za minimizaciju).
- Posebni slučajevi: neograničen problem, nedopustiv problem, degeneracija (više baza za isto teme), alternativni optimumi.

Dualni problem

Za primal $\min c^T x, Ax = b, x \geq 0$ dual (D) je:

$\max b^T y$, uz $A^T y \leq c, y$ slobodno

Pravila za konstrukciju duala iz opšteg oblika (min problem):

Primal (min)	->	Dual (max)	Primal (min)	->	Dual (max)
ograničenje $\geq$		promenljiva $y \geq 0$	promenljiva $x \geq 0$		ograničenje $\leq$
ograničenje $\leq$		promenljiva $y \leq 0$	promenljiva $x \leq 0$		ograničenje $\geq$
ograničenje $=$		y slobodno	promenljiva x slobodna		ograničenje $=$

Ključne teoreme

1. **Slaba dualnost:** za bilo koje dopustivo x i y važi $b^T y \leq c^T x$. Dual daje donju granicu primala.
2. **Jaka dualnost:** ako jedan problem ima konačan optimum, ima ga i drugi, i optimalne vrednosti su **jednake**.
3. **Komplementarna labavost:** x i y su optimalni akko je $x_j \cdot (c_j - A_j^T y) = 0$ za svako j (i analogno za ograničenja kod nejednakosti). Ako ograničenje nije aktivno, njegova dualna promenljiva je 0. Ako je dualno ograničenje labavo, odgovarajuća primalna promenljiva je 0.
4. **Farkasova lema:** osnova za dokaz nedopustivosti (sertifikat).

Analiza osetljivosti

Pitaš koliko se parametri smeju promeniti a da optimalna **baza** ostane ista:

- **Opseg za c_j :** u kom intervalu cena rešenje ostaje optimalno.
- **Opseg za b_i :** u kom intervalu je dualna promenljiva važeća (baza ostaje dopustiva, $B^{-1}b \geq 0$).
- Dodavanje nove promenljive ili ograničenja: proveriti reducirani trošak, odnosno dopustivost, bez ponovnog rešavanja od nule.

Praktični zadatak 1: Fabrika nameštaja

Fabrika proizvodi **stolove, stolice i police**. Cilj je maksimizovati dnevni profit.

	Sto	Stolica	Polica	Raspoloživo
Drvo (m ³)	3	1	2	240
Stolarija (h)	4	2	3	360
Farbanje (h)	2	1	2	180
Pakovanje (h)	1	1	1	100
Profit (€/kom)	120	70	90	

Dodatna ograničenja:

- Ugovor obavezuje fabriku da proizvede **najmanje 10 stolica**.
- Tržište upija **najviše 40 police**.
- Količine mogu biti razlomljene (to je LP, ne celobrojni problem).

Definicija problema i koda

Ciljna funkcija: $\max \{120X_1 + 70X_2 + 90X_3\}$

Ograničenja ($A_{ub} \cdot x \leq b_{ub}$):

$$3X_1 + 1X_2 + 2X_3 \leq 240$$

$$4X_1 + 2X_2 + 3X_3 \leq 360$$

$$2X_1 + 1X_2 + 2X_3 \leq 180$$

$$1X_1 + 1X_2 + 1X_3 \leq 100$$

Granice (bounds):

$$X_1, X_2, X_3 \geq 0$$

$$X_2 \geq 10$$

$$X_3 \leq 40$$

Ovo predstavljamo u matričnom obliku u kodu koristeći NumPy biblioteku.

Matrica ograničenja
koeficijenti uz x (**A_ub**)

```
A_ub = np.array(  
[[3, 1, 2],  
[4, 2, 3],  
[2, 1, 2],  
[1, 1, 1]])
```

Vektor ograničenja
desne strane (**b_ub**)

```
b_ub = np.array(  
[240,  
360,  
180,  
100])
```

Koeficijenti ciljne funkcije
(**c**)

```
c = np.array(  
[-120,  
-70,  
-90])
```

! Zbog **linprog** funkcije
biblioteke **SciPy** koja očekuje
problem transformisan u
minimizaciju koeficijenti su
negativni.

Ciljna funkcija postaje:
 $\min \{-120X_1 - 70X_2 - 90X_3\}$

Uz **A_ub**, **b_ub**, **c** potrebne su nam i granice (**bounds**), i algoritam koji ćemo koristiti (**method**).

- **Granice** za X2 imamo u definiciji problema ($X_2 \geq 10$), takođe i za X3 ($X_3 \leq 40$), za X1 određujemo da će biti pozitivan jer je nerealno imati negativan broj stolova.
Znači: **bounds** = [(0, None), (10, None), (0, 40)].
- **Algoritam** koji koristim, koji je brz i pouzdan jeste HiGHS.
Znači: `method="highs"`.

Nakon ove transformacije i predstavljanja problema preko matričnog sistema imamo sve što nam je potrebno za izvršavanje **linprog** funkcije.

```
result = linprog(c, A_ub=A_ub, b_ub=b_ub, bounds=bounds, method="highs")
```

Ukoliko je uspešno izvršena funkcija (proveravamo **result.success**):

Koristimo **result.fun** za ispis optimalnog rešenja ciljne funkcije, odnosno maksimalni profit.
Koristimo **result.x** nam za ispis X1,X2,X3 vrednosti (broj stolova, stolica, polica koji je proizveden pod datim uslovima za maksimalni profit).

Rezultat izvršavanja:

Github link koda ([factory.py](#))

Ukupni maksimalni profit za dan: 10500.00 €

Proizvedeno stolova: 70.00

Proizvedeno stolica: 30.00

Proizvedeno polica: 0.00

Možemo proveriti profit $120 \cdot 70 + 30 \cdot 70 + 0 \cdot 90 = 10500$

Slack:

Slack je neiskorišćeni deo ograničenja. Za ograničenje oblika $a \cdot x \leq b$ slack je razlika između onoga što je dozvoljeno i onoga što optimalno rešenje zaista koristi:

$\text{slack} = b - a \cdot x$, linprog je takođe izračunao slack i njegovu vrednost ispisujemo koristeći `result.slack`. Slack može biti jednak nuli ili veći od nule, ukoliko je slack za neko ograničenje nula, to ograničenje je **aktivno**, ukoliko ograničenje nije aktivno, odnosno veće je od nule dodatna količina tog resursa, koji odgovara tom ograničenju, ne bi donela ništa.

`result.slack = [0, 20, 10, 0]`

drvo	slack = 0.00 -> AKTIVNO
stolarija	slack = 20.00 -> ima viška
farbanje	slack = 10.00 -> ima viška
pakovanje	slack = 0.00 -> AKTIVNO

Shadow prices i marginale granica

Naučio sam kako određujemo i **shadow prices i marginale granica**, i šta su:

Pokazatelji koliko se promeni optimalna vrednost cilja kad za jednu jedinicu "olabavimo" određeno ograničenje ili granicu.

Koristeći `result.ineqlin.marginals`, `result.lower.marginals`, `result.upper.marginals`

shadow prices: [-25. -0. -0. -45.]

reducirani troškovi (donje granice): [0. 0. 5.]

reducirani troškovi (gornje granice): [0. 0. 0.]

1. Shadow prices: [-25, 0, 0, -45]

Drvo: -25 znači da bi jedan dodatni m³ drveta povećao dnevni profit za 25 €.

Pakovanje: -45 znači da bi jedan dodatni sat pakovanja povećao profit za 45 €.

Stolarija i farbanje: 0 jer imaju slack (20 i 10), pa dodatni sati ne bi ništa doneli.

2. Donje granice: [0, 0, 5]

Redom su to sto, stolica, polica.

Sto: 0. Proizvodi se 70 komada, daleko od donje granice 0, pa granica nije aktivna.

Stolica: 0. Proizvodi se 30, a granica je 10. Ugovor te ne ograničava. Da je obaveza bila 30 ili manje, rešenje bi bilo isto.

Polica: 5. Polica je tačno na donjoj granici (0), i to je njen reducirani trošak. Kad bi silom napravio jednu policu, minimizacioni cilj bi porastao za 5, odnosno profit bi pao za 5 €.

Provera: polica troši 2 m³ drveta i 1 h pakovanja, što po shadow price-u vredi $2 \cdot 25 + 1 \cdot 45 = 95$ €. Njen profit je samo 90 €, pa je razlika -5 €. Polica bi ušla u plan tek kad bi njen profit porastao preko **95 €/kom.**

3. Gornje granice: [0, 0, 0]

Nijedna promenljiva ne stoji na gornjoj granici. Za policu (≤ 40) je proizvodnja 0, što je daleko od 40, pa povećanje tržišta polica ne bi ništa promenilo. Gornji marginal nije nula tek kad je promenljiva "prikucana" uz gornju granicu, i tada bi u max problemu značio koliko vredi dodatna jedinica tržišta.

Dual

- svako ograničenje primala daje jednu dualnu promenljivu ≥ 0
- svaka promenljiva primala daje jedno ograničenje duala
- primal je max, dual je min

Primalni ima 4 resursa, a uslovi $x_2 \geq 10$ i $x_3 \leq 40$ se tretiraju kao ograničenja ($-x_2 \leq -10$ i $x_3 \leq 40$). Dual u tom slučaju ima 6 promenljivih: $y_1 \dots y_4$ (resursi), w (za stolice) i v (za police).

$$\min \quad 240y_1 + 360y_2 + 180y_3 + 100y_4 - 10w + 40v$$

$$\begin{aligned} \text{uz} \quad & 3y_1 + 4y_2 + 2y_3 + y_4 \geq 120 \quad (\text{sto}) \\ & y_1 + 2y_2 + y_3 + y_4 - w \geq 70 \quad (\text{stolica}) \\ & 2y_1 + 3y_2 + 2y_3 + y_4 + v \geq 90 \quad (\text{polica}) \end{aligned}$$

$$y_1, y_2, y_3, y_4, w, v \geq 0$$

Desne strane primala (240, 360, 180, 100, -10, 40) postaju koeficijenti dualnog cilja. Zato ide $-10w$.

Dualni uslovi su $\geq$, a **linprog** očekuje $\leq$. Pomnoži ih sa -1 .

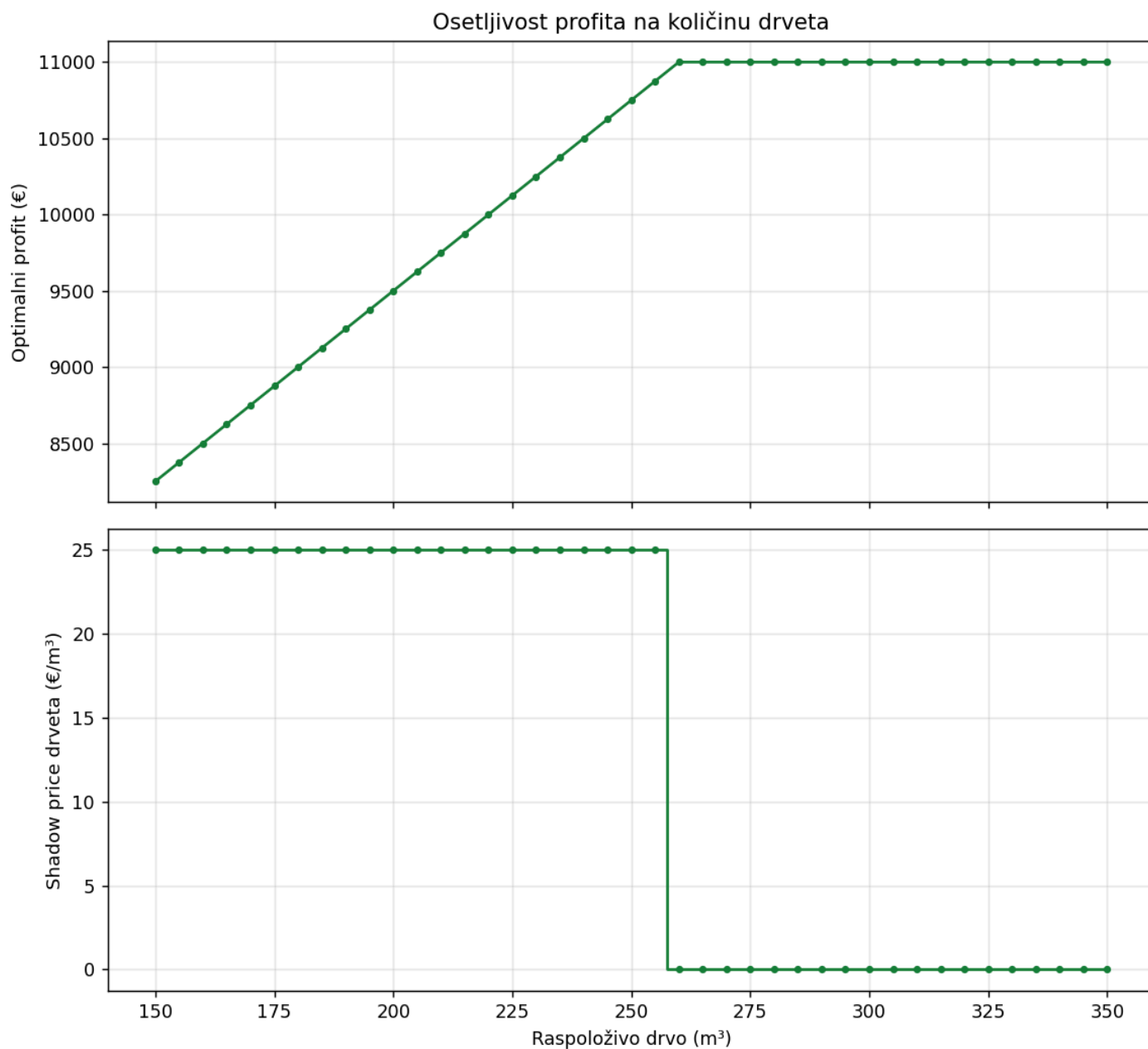
Provera: Provera ovakvog sistema u kodu dovodi do istog optimalnog rešenja, 10500.

Možemo koristiti i **OR-Tools GLOP** za rešavanje ovog problema ovaj model je zadat kao max pa je znak dualnih vrednosti (shadow prices pozitivan provereno u kodu)

Github link koda ([factoryGLOP.py](#))

DODATNO: Vizuelizacija sa različitim zahtevima, uz pomoć **matplotlib**

Github link ka kodu ([visualisation.py](#))



Za svaku vrednost drveta od 150 do 350 (korak 5) rešava se LP, a pamte se optimalni profit i shadow price drveta (sa obrnutim znakom, jer je max $\rightarrow$ min).

Drvo (m ³)	Profit (€)	Shadow price (€/m ³)
150	8 250	25
200	9 500	25
240	10 500	25
255	10 875	25
260	11 000	0
350	11 000	0

Profit je **po delovima linearan** i ima jedan prelom:

- **150 do 260 m³:** profit raste sa nagibom **25 €/m³**, što je tačno shadow price.
Provera: $(11\,000 - 10\,500) / (260 - 240) = 25$.
- **Iznad 260 m³:** profit je konstantan, 11 000 €, nagib 0.

Zašto se nagib lomi baš na 260

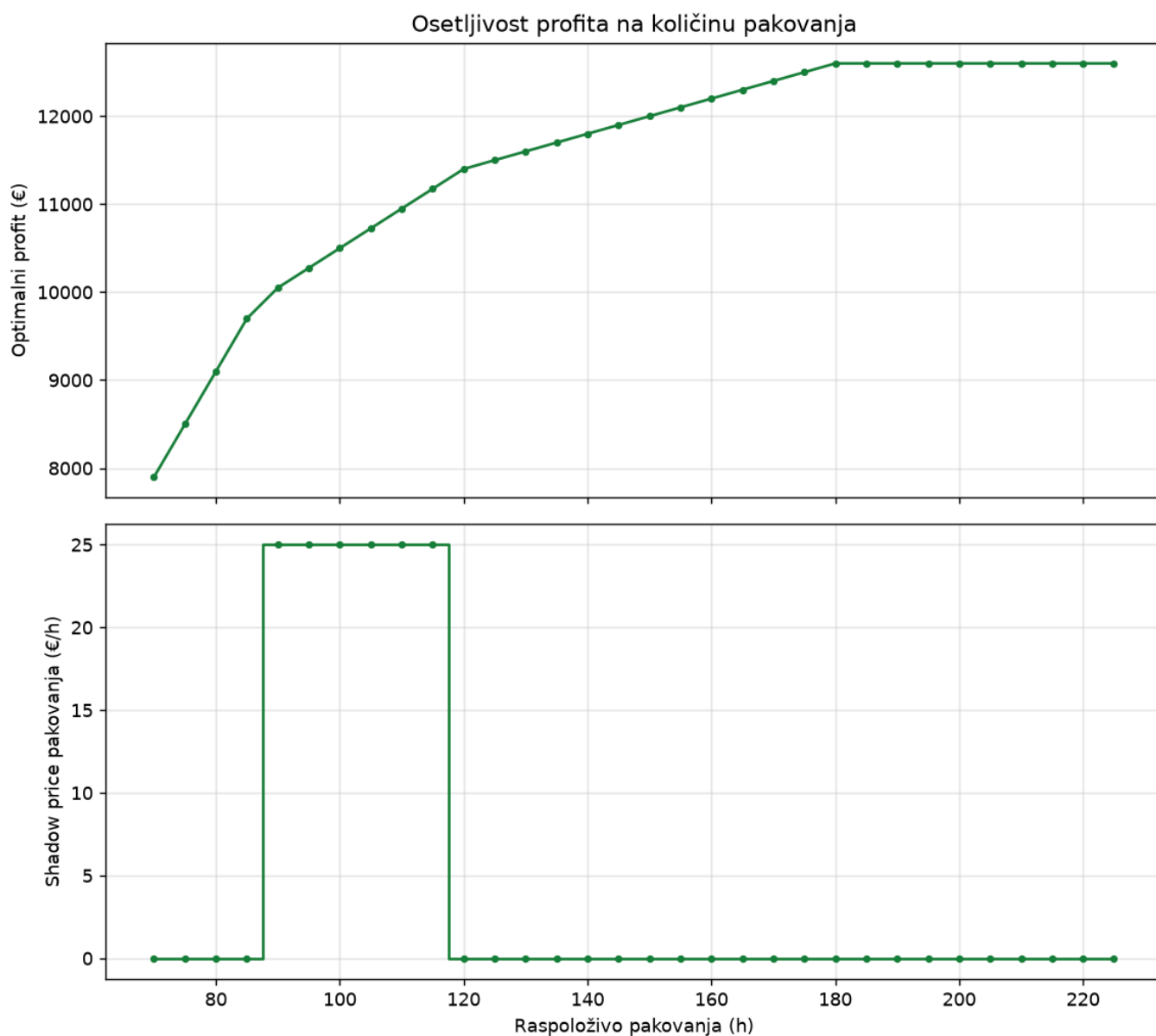
Dok vrednosti drveta rastu, mogu da se prave sve veće količine stolova. Do 260 su aktivni **drvo i pakovanje**. Pri 260 m³ rešenje postaje $x = (80, 20, 0)$:

- stolarija: $4 \cdot 80 + 2 \cdot 20 = 360$, dostignuta je granica
- pakovanje: $80 + 20 = 100$, i dalje aktivno
- drvo: $3 \cdot 80 + 20 = 260$, ali sada više nije usko grlo

Od te tačke **stolarija i pakovanje** zajedno drže rešenje u temenu (80, 20, 0), pa dodatno drvo ne pomaže. Usko grlo se "preselilo" sa drveta na stolariju, a baza se promenila.

Tu se vidi zašto je shadow price važeći samo u opsegu: za drvo od otprilike 100 do 260 vredi 25 €/m³, a iznad toga vredi 0. Da je cena drveta na tržištu 20 €/m³, isplatilo bi se dokupiti do 260 m³, a ne više.

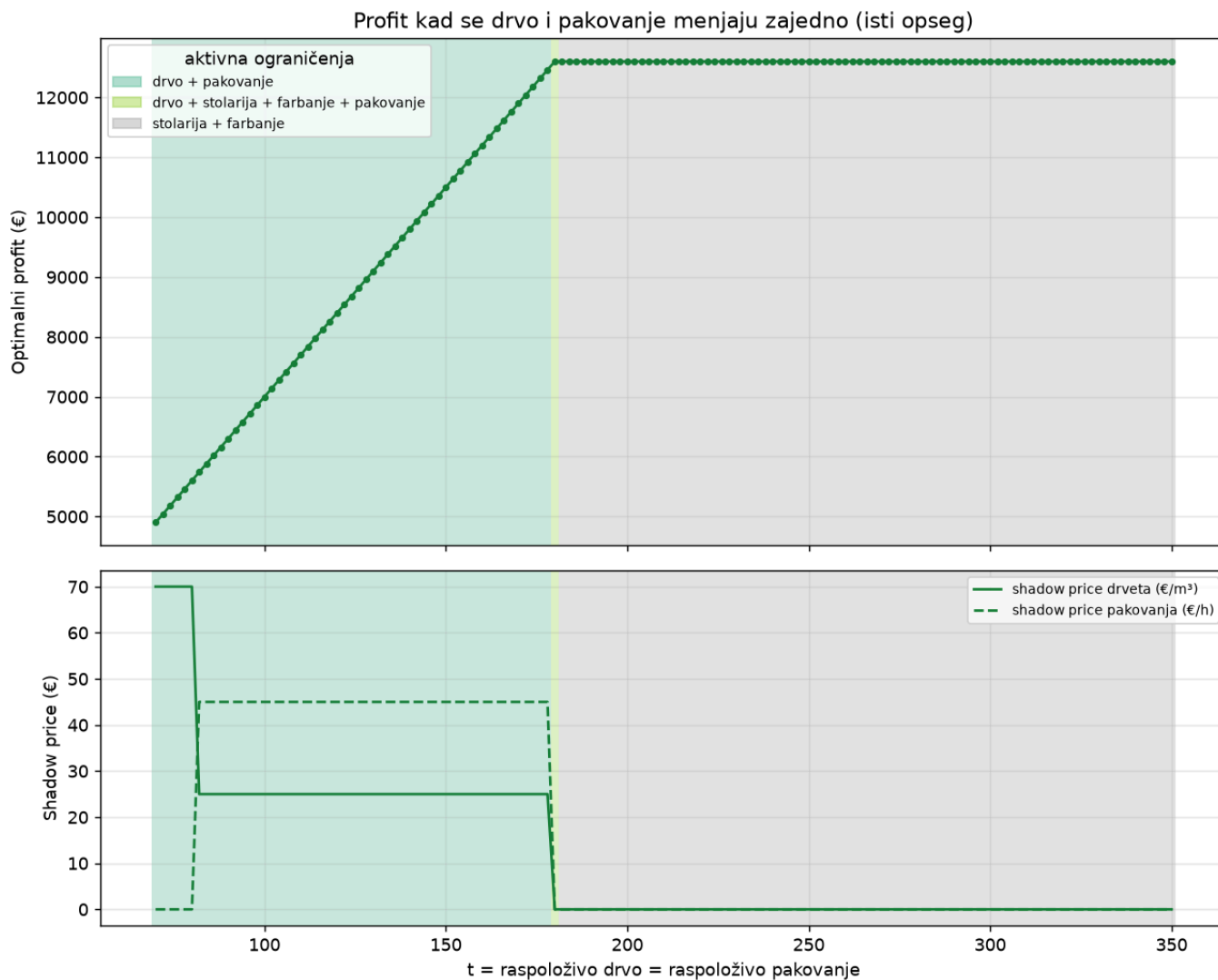
Primer vizuelizacije, osetljivosti ograničenja, ukoliko povećavamo sate pakovanja



Profit ostaje nepromenjen nakon 12600€

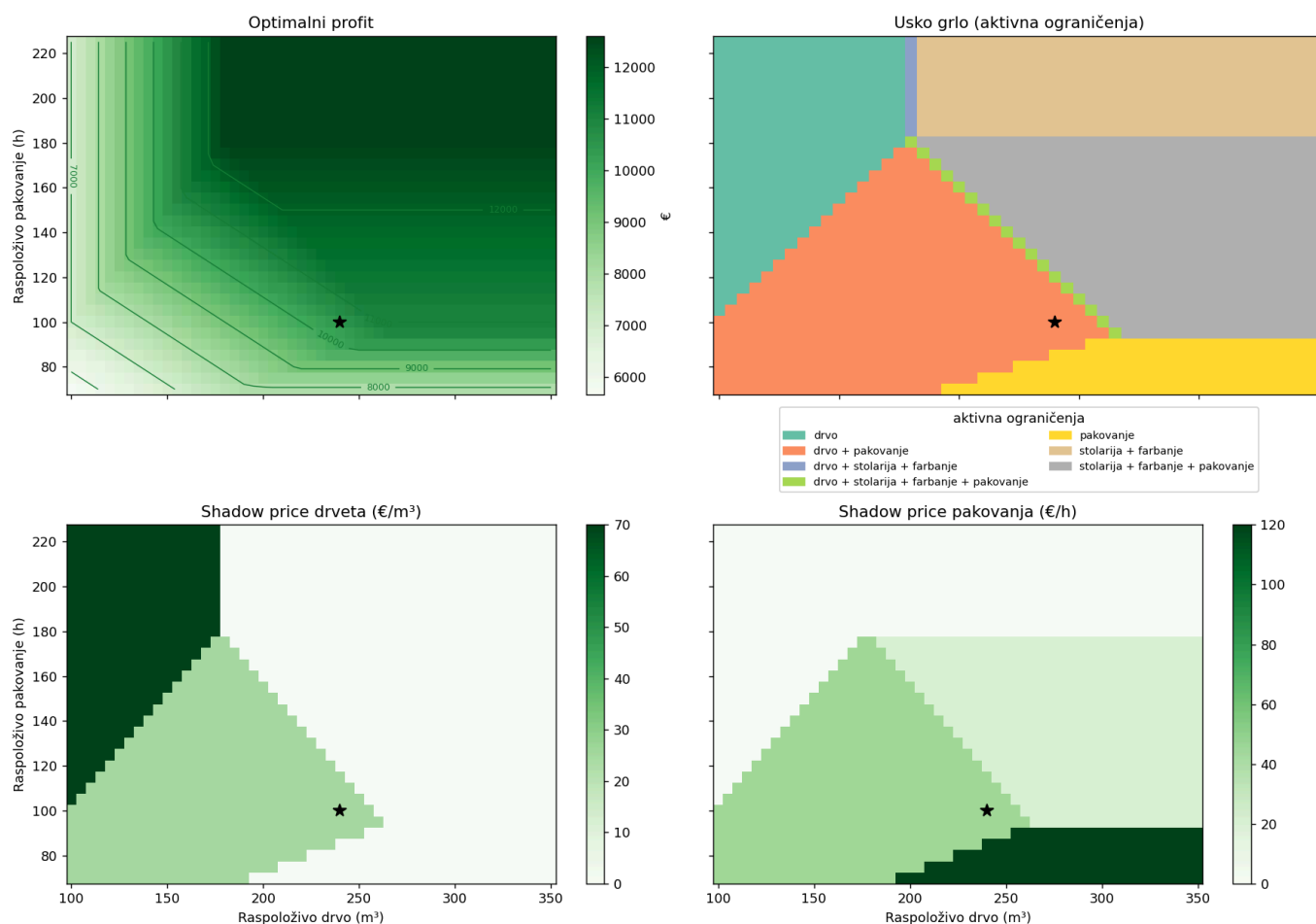
Primer vizualizacije, osetljivosti ograničenja, ukoliko povećavamo u isto vreme različita ograničenja za istu vrednost

Za menjanje vrednosti izabraćemo i drvo i pakovanje, jer su nam to bila 2 uska grla (da vidimo šta se onda dešava)



Primer vizualizacije, osetljivosti ograničenja, ukoliko povećavamo u isto vreme različita ograničenja za različite vrednosti

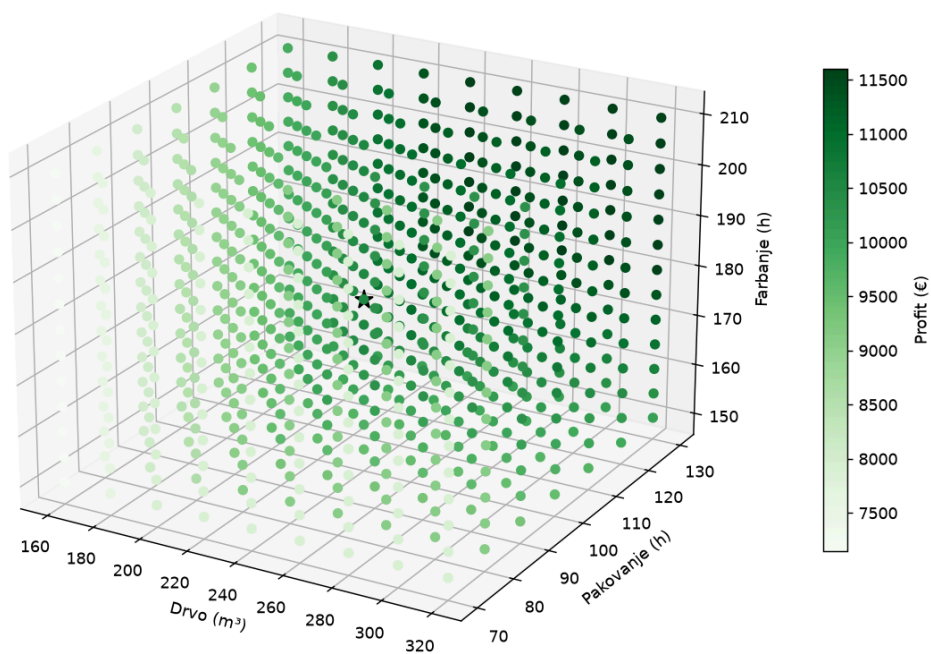
Za menjanje vrednosti izabraćemo i drvo i pakovanje, jer su nam to bila 2 uska grla (da vidimo šta se onda dešava). Ukoliko menjamo ograničenja za različite vrednosti dobijamo 2D-graf koji prikazuje promene. Ovo istražujem uz pomoć LLM alata, moje veštine programiranja grafova su još nedovoljno razvijene.



Primer vizualizacije, osetljivosti ograničenja, ukoliko povećavamo u isto vreme 3 različita ograničenja za 3 različite vrednosti (3D graf)

Drvo, pakovanje i farbanje se menjaju zajedno (stolarija fiksirana na 360)

Optimalni profit



Usko grlo

